

Reviewer Pack — Khinchin’s Constant Bounds Communication Complexity of Boole...

Entry ca5253051c60 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 02:32:47 UTC

Khinchin’s Constant Bounds Communication Complexity of Boolean Functions

Entry ID: ca5253051c60 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 02:32:47 UTC

1. Conjecture

Field A (mathematical branch): Statistical mechanics (specifically, ergodic theory) **Field B** (complexity object): Communication complexity

Statement:

For any boolean function f with n inputs, the communication complexity $CC(f)$ is bounded by the Khinchin’s constant $K_2^{(n-1)}$, where $K_2 \approx 0.41421356237309504$, and this bound is tight up to a constant factor.

Rationale (proposer’s reasoning):

Khinchin’s constant arises in ergodic theory and is related to the statistical behavior of deterministic processes. By connecting it to communication complexity, we may uncover new insights into the computational aspects of information transfer.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3f9912ca98d54f3c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each seed, if the ratio of the communication complexity $CC(f)$ to Khinchin’s constant raised to the power $(n-1)$ is within $\pm 5\%$ of 1 for all generated boolean functions f with n inputs ($n \leq 40$), then support is provided for the conjecture. Falsification occurs if this ratio exceeds $\pm 5\%$ of 1 for any seed or function.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.80	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Khinchin's constant" AND "communication complexity" AND boolean function" - "ergodic theory" AND "Boolean functions" AND communication complexity" - "statistical mechanics" AND "Communication complexity" AND "Khinchin's constant"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

# Khinchin's constant  $K_2 \approx 0.41421356237309504$ 
K_2 = 0.41421356237309504

def communication_complexity(f):
    # Generate a random input vector of length n
    n = len(f)
    x = [random.choice([0, 1]) for _ in range(n)]

    # Evaluate the function on the input vector
    y = f(x)

    # Calculate the communication complexity
    CC_f = sum(1 if xi != yi else 0 for xi, yi in zip(x, y))

    return CC_f

def run_trial(seed: int) -> dict:
    random.seed(seed)

    metric_name = "communication_complexity"
    instances_tested = 30
    n_max = 40
    conjecture_holds = True
    counterexample = ""

    total_CC_f = 0

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5): # Generate 5 instances per size
            f = lambda x: [x[i] ^ x[(i + 1) % n] for i in range(n)]
            CC_f = communication_complexity(f)
            total_CC_f += CC_f
```

```

mean_CC_f = total_CC_f / (instances_tested * n_max)
ratio = mean_CC_f / (K_2 ** (n_max - 1))

if abs(ratio - 1) > 0.05:
    conjecture_holds = False
    counterexample = f"Ratio {ratio} outside ±5% of 1"

return {
    "metric_name": metric_name,
    "metric_value": mean_CC_f,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97]

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next((seed for seed, res in zip(seeds, results) if not res["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"{res['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_alald352.py", line 73, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_alald352.py", line 48, in run_trial

```

    CC_f = communication_complexity(f)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_alald352.py", line 23, in communication_complexity

```

    n = len(f)

```

^^^^^

TypeError: object of type 'function' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Review and debug the test code to ensure it can run successfully and produce results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13576
2	preregistration	ollama_remote	glm4:latest	0	0	10140
3	novelty	ollama_remote	glm4:latest	0	0	8357
4	novelty	ollama_remote	glm4:latest	0	0	8841
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13417
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7992
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9236
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17995
9	judge	ollama_remote	glm4:latest	0	0	10133

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 99686 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/ca5253051c60.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/ca5253051c60.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/ca5253051c60.tar.gz* (if generated)